

Genetic Algorithm for Manufacturing-Parameter Optimization

Optimizing the temperature, pressure and material ratio of a recycled rubber-plastic composite casing using a standard Genetic Algorithm

Particulars	Details
Name of the student	
Enrolment No. · Branch	Robotics and Automation
Subject / Activity	Problem-Based Learning Activity 2 — Genetic Algorithm
Date	
Sign of Faculty · Date of Evaluation	

Aim

To formulate and structure a standard **Genetic Algorithm (GA)** to optimize the manufacturing parameters — **melting temperature, injection pressure and rubber-to-plastic ratio** — of a recycled rubber-plastic composite casing, so that the optimal recipe is found digitally before any physical production begins.

Relevant Theory (in brief)

A **Genetic Algorithm** is a heuristic search and optimization technique based on Charles Darwin's theory of natural selection — "survival of the fittest". A **population** of candidate solutions (called **chromosomes**) is evolved over successive **generations**. Each chromosome is scored by a **fitness function**; the fitter chromosomes are **selected** as parents, combined by **crossover** and altered by **mutation** to produce the next generation. The cycle repeats until **convergence**. The five basic operations are **Encoding, Fitness Evaluation, Selection (Reproduction), Crossover and Mutation**.

Note — in this problem a "chromosome" is one complete manufacturing **recipe**, and its "fitness" is the **strength/quality of the casing** that recipe would produce.

Task 1 — System Encoding & Fitness Definition (4 Marks)

1.1 Construct a Chromosome (Value Encoding)

In **value encoding**, each gene directly stores the real value of a process parameter (not a binary string). The chromosome is a 3-gene array, one gene per manufacturing parameter, with realistic working ranges:

Gene	Parameter	Realistic range	Why this range
Gene 1	Rubber Percentage (%)	10 – 40 %	Below 10 % → brittle; above 40 % → casing too soft and fails.
Gene 2	Melting Temperature (°C)	180 – 250 °C	Below 180 °C → plastic not fully melted; above 250 °C → rubber degrades.
Gene 3	Injection Pressure (Bar)	600 – 1200 Bar	Below 600 Bar → mold not filled (short shot); very high → flash/waste.

One example of a complete chromosome (one recipe):

```

      Gene 1      Gene 2      Gene 3
      Rubber %    Temp (C)    Pressure (Bar)
[    30    ,    220    ,    900    ]

Chromosome = [ 30, 220, 900 ]

```

This recipe means: **30 % rubber, melted at 220 °C, injected at 900 Bar.**

1.2 Define the Fitness Function

The fitness function gives each recipe a **Casing Strength Score (S)** — higher is better. It rewards good shock absorption and complete mold filling, and heavily penalises any recipe that violates a safe-process limit. A representative formulation:

```

Fitness S = IS + FF          (reward part)

IS (Impact-strength term) = 50 - 2 x | R - 30 |
    -> maximum (50) when rubber R = 30 %, i.e. best shock absorption;
    falls as the recipe moves away from the ideal rubber content.

FF (mold Fill-quality term) = P / 20
    -> rises with injection pressure P (better, complete filling).

where R = Rubber % (Gene 1), T = Temp (Gene 2), P = Pressure (Gene 3).

```

High Fitness Score condition (reward): a recipe scores high when it gives **maximum impact strength** (best shock absorption, rubber near the ideal ~30 %) **and** the **mold fills completely** (sufficient pressure) **and** the hardness stays within the acceptable band.

Penalty condition (score = 0, recipe rejected): the recipe is infeasible and its fitness is forced to **S = 0** if any safe limit is crossed —

- **T > 250 °C** → rubber thermally **degrades** → S = 0
- **P < 600 Bar** → mold not filled (**short shot**) → S = 0
- **R > 40 %** → casing **too soft**, fails the load test → S = 0
- **T < 180 °C** → plastic not fully melted → S = 0

Worked example — scoring the chromosome [30, 220, 900]:

```

R = 30, T = 220, P = 900
Check penalties: T=220 (<=250 ok), P=900 (>=600 ok), R=30 (<=40 ok) -> feasible

IS = 50 - 2 x |30 - 30| = 50 - 0 = 50
FF = 900 / 20 = 45
Fitness S = 50 + 45 = 95 (out of ~100) -> a strong recipe.

```

Task 2 — Execution of Genetic Operations (5 Marks)

2.1 Reproduction (Selection Method)

Selection method chosen: Tournament Selection. A small group of $k = 3$ recipes is picked at random from the population, their fitness scores are compared, and the one with the **highest Casing Strength Score becomes a parent**. The tournament is repeated until the mating pool is full.

Why Tournament Selection: it is simple and fast, it does not need the fitness values to be scaled (unlike Roulette Wheel selection), it still gives weaker recipes a small chance to compete (maintaining diversity), and its selection pressure is easy to tune by changing the tournament size k . *(Roulette-Wheel selection, where each recipe is picked with probability proportional to its fitness, is the common alternative.)*

2.2 Crossover — Single-Point Crossover (Dry Run)

Two selected parent recipes are combined by cutting both at the **same single point** and swapping the tail segments. Here the crossover point is taken **after Gene 2** (so Genes 1 & 2 stay together and Gene 3 is exchanged):

```
Crossover point | is after Gene 2

Parent 1 = [ 30 , 220 | 900 ]
Parent 2 = [ 20 , 240 | 750 ]
----- swap the part after the point -----
Offspring 1 = [ 30 , 220 | 750 ]      (Genes 1,2 from P1 ; Gene 3 from P2)
Offspring 2 = [ 20 , 240 | 900 ]      (Genes 1,2 from P2 ; Gene 3 from P1)
```

The two **resulting offspring chromosomes** are therefore **O1 = [30, 220, 750]** and **O2 = [20, 240, 900]** — new recipes that mix the "good parts" of both parents.

2.3 Mutation

Take Offspring 1, **O1 = [30, 220, 750]**. Apply a mutation operator by altering **one gene** — change the injection pressure by a small random amount, **+30 Bar**:

```
Before mutation : O1 = [ 30 , 220 , 750 ]
                  ^ mutate Gene 3 (pressure): 750 -> 780
After mutation  : O1' = [ 30 , 220 , 780 ]
```

Why mutation is necessary for the factory process: mutation injects **new genetic diversity** that was not present in the parent recipes, so the GA can explore pressure/temperature/rubber combinations that no existing recipe had. This prevents the algorithm from getting **stuck in a local optimum (premature convergence)** — i.e. from repeatedly recombining the same few recipes and concluding too early. In factory terms, it lets the search **discover a genuinely new and possibly better recipe** (for example a slightly higher pressure that fills the mold better) and keeps the whole parameter space reachable, instead of just averaging the recipes already tried.

Task 3 — Generational Cycle & Convergence (3 Marks)

3.1 Flowchart — GA Generational Cycle

The standard cycle (Evaluation → Selection → Crossover → Mutation → Replacement) is shown below with the blocks labelled in manufacturing terms:

GA Generational Cycle — Rubber-Plastic Casing Recipe Optimisation

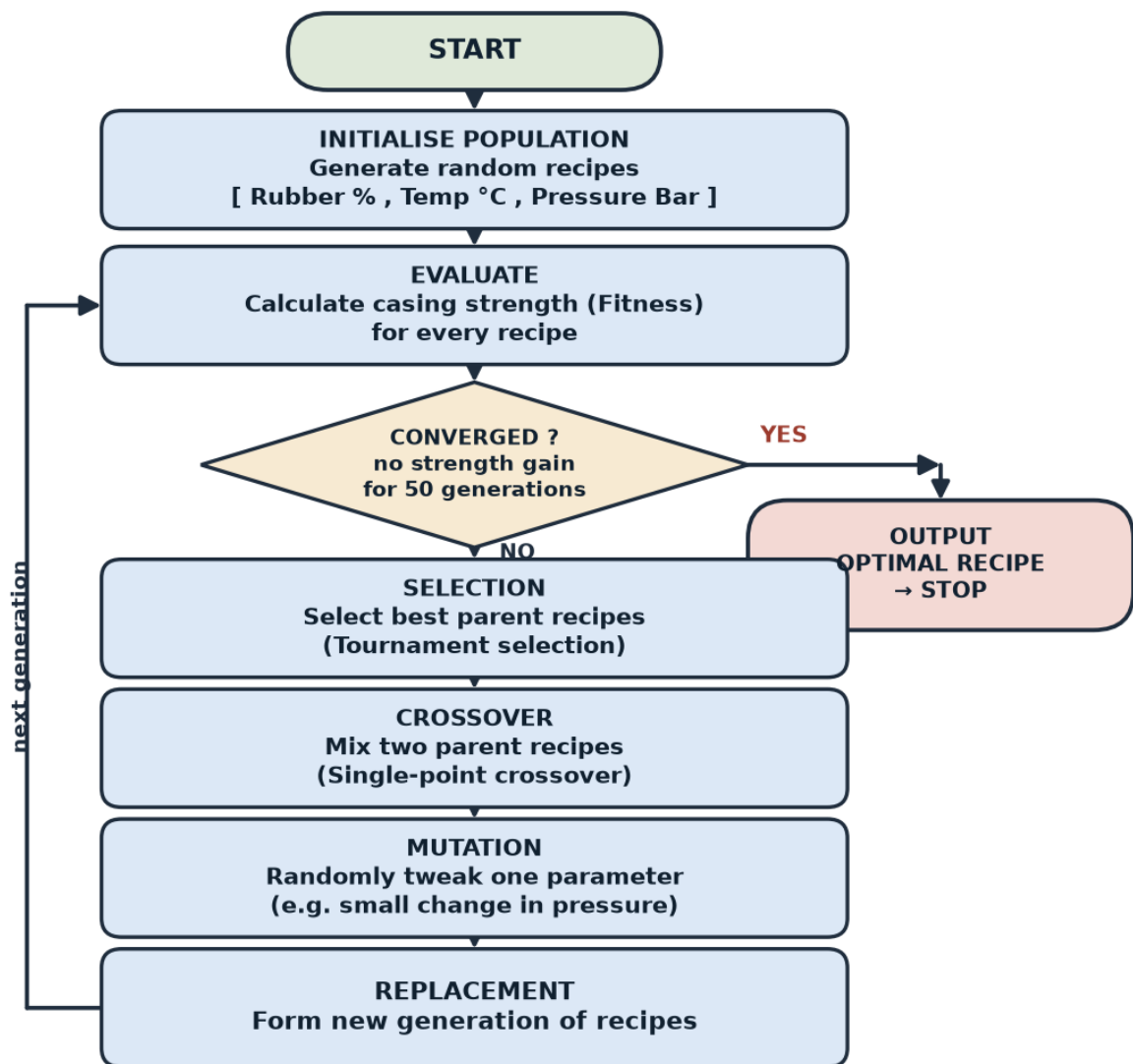


Fig. 1 — Genetic Algorithm generational cycle for the rubber-plastic casing recipe. The loop repeats generation after generation until the convergence (stopping) criterion is met.

3.2 Define Convergence (Stopping Criteria)

Convergence is the point at which the GA stops looping because it has effectively found the best recipe. The machine **stops when the first of these conditions is met**:

1. **No improvement (stagnation)**: the best Casing Strength Score does **not improve (change < 1 %)** for **50 consecutive generations** — the main criterion.
2. **Maximum generations**: a hard limit, e.g. **200 generations**, is reached (safety stop).
3. **Target reached**: the best fitness reaches the design target, e.g. **$S \geq 95$** , i.e. the required casing strength is achieved.
4. **Population converged**: nearly all recipes in the population have become almost identical (diversity falls below a set threshold).

Exact stopping rule used here — "The GA stops when the best casing-strength score shows **no improvement for 50 consecutive generations**, OR a maximum of **200 generations** is reached — whichever comes first. The recipe with the highest fitness at that moment is declared the **optimal manufacturing recipe**."

Task 4 — Comparative Analysis (3 Marks)

4.1 GA vs Traditional Optimization Method

Point of comparison	Genetic Algorithm (GA)	Traditional method (manual trial-and-error / gradient descent)
Search approach	Population-based — explores many recipes in parallel each generation	Tests one recipe at a time (trial-and-error) or follows a single path (gradient descent)
Global vs local optimum	Escapes local optima through crossover and mutation → finds a near-global optimum	Easily trapped in a local optimum ; trial-and-error gives no guarantee at all
Need for a derivative	Needs no gradient/derivative — works with discontinuous, penalty-based fitness	Gradient descent needs a smooth, differentiable function; not available here
Raw-material cost	Runs digitally — no physical material wasted while searching	Trial-and-error wastes raw material, molds and time on every failed batch
Handling many interacting parameters & hard limits	Handles the non-linear interaction of rubber, temperature and pressure and the reject limits with ease	Becomes very difficult as the three parameters interact and constraints collide

4.2 Why GA is better suited to this rubber-plastic process

- The relationship between rubber %, temperature, pressure and casing strength is **non-linear and strongly interacting**, and the hard limits (rubber degradation, short shot, excessive softness) create **sudden "fitness cliffs"** — a rugged, discontinuous search landscape with many local optima.
- Gradient-based methods **cannot work** on such a landscape because there is no usable derivative across the penalty jumps, and they get stuck in local optima; manual trial-and-error **wastes expensive recycled material** and may never reach the true optimum among the huge number of continuous parameter combinations.
- The GA **searches many recipes at once, purely in software**, naturally respects the "reject" penalty conditions, and steadily evolves toward the best recipe — so it finds a near-optimal temperature/pressure/rubber combination **before any material is consumed in production**, which is exactly the objective of this problem.

Conclusion

A complete theoretical framework of a standard Genetic Algorithm was formulated for the recycled rubber-plastic casing. A recipe was **value-encoded** as the 3-gene chromosome [Rubber %, Temp, Pressure]; a **fitness function** was defined that rewards impact strength and mold filling while rejecting unsafe recipes; the **selection (tournament), single-point crossover and mutation** operators were demonstrated on sample recipes; the **generational cycle and convergence criteria** were specified; and GA was shown to be better suited than traditional methods for this non-linear, constrained manufacturing problem. The GA can thus find the optimal manufacturing recipe digitally, saving raw material and time before production begins.